

计算机安全与保密技术课程成绩评价表

课程名称	计算机安全与保密技术	课程编号	08A65002
论文题目	基于 Socket 的 Web 代理服务器设计与实现		
学生姓名		学号	
所在院系			

课程论文评阅记录

课程论文评价要素				
项目设计 35%	技术正确	内容完成	算法新颖	得分：
程序实现 35%	功能完成	运行稳定	设计合理	得分：
报告内容与格式 30%	报告清晰	见解独到	格式规范	得分：
报告值得肯定之处				
报告存在主要问题				
其他需要说明情况				

论文成绩：_____（百分制） 任课教师：_____

（课程论文占期末总评成绩的 60%）

评阅日期：_____

注：后接课程论文，格式参照课程论文模板或公开发表论文的风格。

基于 Socket 的 Web 代理服务器设计与实现

学生姓名 学号

(上海大学, 上海 200444)

摘要: 为实现课程要求的 Web 请求转发、指定域名拦截和网页关键词过滤, 本文设计并实现了一套基于 Python Socket 的正向代理服务器。系统采用多线程处理客户端连接, 通过解析 HTTP 代理请求建立上游连接, 并按照客户端地址、认证状态、访问频率、请求方法、域名和 URL 的顺序执行访问控制; 对于明文 HTTP 文本响应, 代理在返回客户端前检查正文关键词。系统同时提供 HTTPS CONNECT 隧道、GET 缓存、动态规则管理、管理前端和日志审计功能。测试采用批量自动验证与真实外网手工验收相结合的方法, Web/代理功能批次 15 项、规则与运行模式批次 7 项均通过, 真实 HTTP 网站的 URL 与正文过滤能够生成明确的 403 拦截页面。结果表明, 该系统完成了题目规定功能, 并具备规则热更新、运行统计和可追溯验收能力。

关键词: Web 代理; Socket; 访问控制; 关键词过滤; 日志审计; 动态规则

Design and Implementation of a Socket-Based Web Proxy Server

STUDENT NAME

Shanghai University, Shanghai 200444, China

Abstract: This project designs and implements a forward Web proxy server using Python sockets. The proxy parses HTTP proxy requests, establishes upstream connections, forwards responses, and applies access-control policies based on client IP addresses, authentication, request rate, HTTP methods, domains, and URL keywords. For plain HTTP text responses, the proxy examines response bodies and replaces blocked pages with an explicit HTTP 403 notice. The system also provides HTTPS CONNECT tunneling, GET caching, runtime rule management, an administration dashboard, and auditable logs. Automated batch tests and manual tests against real public HTTP websites were used for verification. All 15 Web/proxy tests and 7 rule/runtime-mode tests passed. The results show that the system satisfies the course requirements and provides practical rule updates, statistics, and traceable acceptance evidence.

Key words: Web proxy; Socket; access control; keyword filtering; audit log; dynamic rule

目录

1	课题背景与功能分析	4
1.1	开发目的	4
1.2	功能需求分析	4
1.3	开发环境与技术路线	4
2	系统总体设计	4
2.1	总体架构	4
2.2	模块组成	5
2.3	数据结构与配置模型	5
2.4	请求处理流程	5
3	核心功能原理、设计与实现	6
3.1	HTTP 正向代理与请求转发原理	6
3.2	访问控制策略链原理	6
3.3	网页正文关键词过滤原理	7
3.4	HTTPS CONNECT 隧道原理	7
3.5	缓存、认证与限流设计	7
3.6	动态规则管理与并发一致性	7
3.7	日志审计与管理前端设计	8
4	测试方案与验收结果	8
4.1	批量验证式测试	8
4.2	手工验收准备	10
4.3	普通转发与 HTTPS 隧道验收	10
4.4	真实网站域名拦截验收	10
4.5	真实网站 URL 关键词验收	11
4.6	真实网站正文关键词验收	12
4.7	缓存、认证、限流与客户端地址策略验收	12
4.8	日志、统计与规则变更验收	15
4.9	测试结果分析	15
5	项目开发总结	16
5.1	完成情况	16
5.2	存在问题与改进方向	16
6	心得体会	16
	参考文献	16
A	关键源代码（带注释）	16
A.1	HTTP 请求解析、改写与转发	16
A.2	访问控制策略判定	19
A.3	HTTP 明文正文关键词过滤	20
A.4	HTTPS CONNECT 双向隧道	22
A.5	缓存、认证与固定时间窗口限流	23

A.6 动态规则与缓存一致性	25
A.7 日志查询与 CSV 审计导出	26
A.8 客户端连接的有序处理总入口	28
B 常用运行与验收命令	31

1 课题背景与功能分析

1.1 开发目的

Web 代理服务器位于客户端和目标 Web 服务器之间。客户端将访问请求发送给代理，代理解析请求中的目标地址，代替客户端连接目标服务器，再把响应转交给客户端。由于请求和响应均经过代理，代理能够在建立上游连接前判断目标是否允许访问，也能够收到明文响应后检查网页正文。该工作过程覆盖了 TCP 连接、HTTP 报文解析、访问控制、内容过滤和安全审计等网络安全基础知识。

本课题的开发目的包括三个方面。第一，完成题目规定的 HTTP 请求转发、指定域名拦截和网页关键词过滤功能。第二，通过 Socket 和 HTTP 协议解析理解正向代理的数据处理过程，并明确普通代理面对 HTTPS 加密流量时的可见性边界。第三，建立可管理、可验证和可审计的实验系统，使规则能够在运行期间修改，使每次放行与拦截都能够通过统计和日志进行说明。

1.2 功能需求分析

项目功能按照课程必做功能、系统管理功能和扩展访问控制功能进行划分。课程必做功能直接对应题目要求；系统管理功能用于解决规则如何修改、结果如何展示、行为如何追溯的问题；扩展访问控制功能用于体现代理服务器在实际网络安全场景中的使用方式。

功能类别	功能说明
代理转发	接收客户端 HTTP 请求，连接目标服务器，将请求和响应完整转发。
域名与 URL 控制	支持完整域名、父域名后缀、显式通配符和 URL 关键词拦截。
正文关键词过滤	检查明文 HTTP 文本响应，命中关键词时返回代理生成的 403 页面。
客户端访问控制	支持 IPv4、IPv6 和 CIDR 网段的客户端黑名单与白名单。
运行策略	支持请求方法过滤、HTTP Basic 代理认证、固定时间窗口限流和 GET 缓存。
规则管理	支持前端、JSON API 和命令行三种管理方式，规则增删改查后立即生效。
展示与审计	展示统计、排行、缓存状态和规则变更，支持结构化日志查询与 CSV 导出。

1.3 开发环境与技术路线

系统运行于 Windows 环境，使用 Visual Studio Code 进行开发，主要编程语言为 Python 3。代理服务使用 socket 建立 TCP 监听与上游连接，使用 threading 为每个客户端连接提供独立处理线程，HTTPS CONNECT 隧道使用 select 实现双向字节转发。管理后端使用 Python 标准库中的 ThreadingHTTPServer 提供页面和 JSON API，管理前端使用 HTML、CSS 与 JavaScript 实现。

配置和规则使用 JSON 文件保存，规则变更使用 JSONL 逐行记录，访问事件写入结构化文本日志。客户端 IP 和 CIDR 使用 ipaddress 模块校验，域名通配符使用 fnmatch 匹配。该技术路线不依赖大型第三方框架，便于在课程报告中说明协议处理和策略判定的底层过程。

2 系统总体设计

2.1 总体架构

系统由代理服务、管理后端、管理前端、配置与日志存储、测试工具五部分组成。代理服务监听 127.0.0.1:8080，处理浏览器或命令行客户端发来的代理请求；管理后端监听 127.0.0.1:8088，向前端和命令行工具提供管理 API。两个服务共享运行状态，规则修改后能够立即影响后续代理请求。

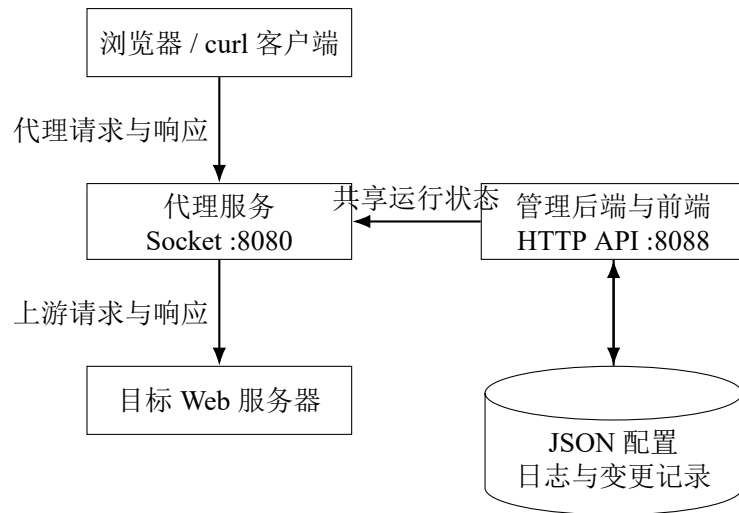


图 1 系统总体架构

2.2 模块组成

代理核心模块负责监听端口、解析 HTTP 请求、执行策略链、转发普通 HTTP 请求和建立 HTTPS CONNECT 隧道。规则配置模块负责读取配置文件、定义规则组、校验并标准化规则值。日志审计模块负责写入访问、拦截和错误事件，并提供查询、清空和 CSV 导出。证据模块负责读取批量测试生成的独立日志，在管理前端中重建最近验收统计。管理前端负责规则编辑、运行设置、统计、日志和规则变更展示。

2.3 数据结构与配置模型

运行状态对象保存当前配置、统计计数、访问排行、关键词命中排行、缓存、限流桶和规则变更历史。共享状态由线程锁保护，避免多个客户端线程和管理线程同时修改数据时产生不一致。

列表型规则包括域名黑名单、域名白名单、客户端 IP 黑名单、客户端 IP 白名单、URL 关键词、正文关键词和禁止方法。运行设置包括黑白名单模式、缓存开关与容量、代理认证、限流上限和时间窗口。规则变更后，系统同时更新内存配置和 JSON 文件，使修改立即生效并在重启后继续保留。

2.4 请求处理流程

请求进入代理后按照固定顺序处理。客户端 IP 规则最先执行，使无权使用代理的主机无法消耗认证和限流资源；认证通过后执行访问频率检查；随后检查请求方法、域名和 URL。前置策略通过后，GET 请求查询缓存，普通 HTTP 请求连接目标服务器并检查响应正文，CONNECT 请求建立加密字节隧道。最后更新统计并写入日志。

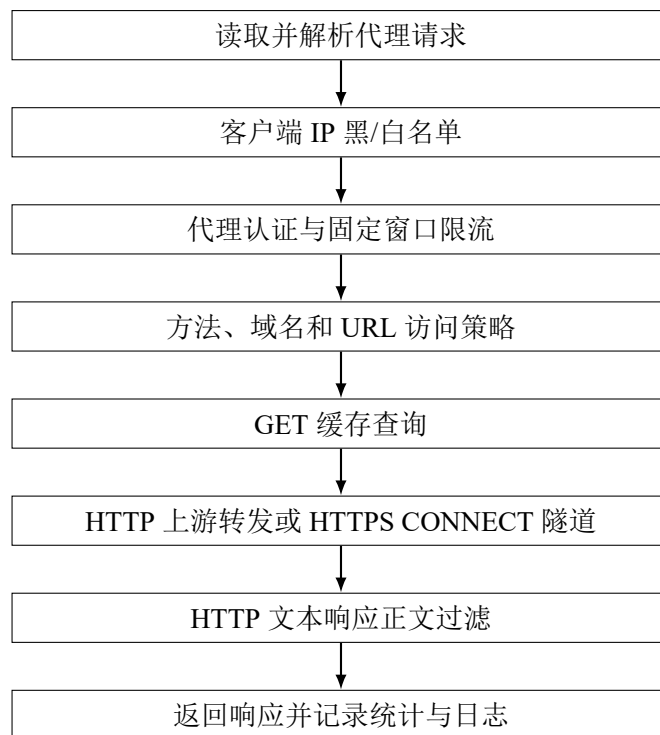


图 2 代理请求处理流程

3 核心功能原理、设计与实现

3.1 HTTP 正向代理与请求转发原理

HTTP 正向代理的核心作用是代替客户端访问目标服务器。客户端直连网站时，请求行通常只包含路径；客户端使用代理时，请求行可以携带完整 URL。代理首先从请求行和 Host 请求头中解析目标主机、端口与路径，再建立到目标服务器的 TCP 连接。由于目标服务器接收的是普通 HTTP 请求，代理需要将完整 URL 改写为路径形式，并重新生成 Host 请求头。

系统在转发前删除 Proxy-Authorization、Proxy-Connection 等只适用于代理连接的请求头，避免代理凭据泄露给目标网站。系统设置 Connection: close，以连接关闭作为本实验中响应读取结束的明确边界；同时设置 Accept-Encoding: identity，使服务器尽量返回未压缩正文，便于关键词检查。

上游连接由 TCP Socket 建立。代理发送改写后的请求后持续读取响应字节，直到目标服务器关闭连接。成功响应经过正文过滤后返回客户端；目标连接失败时返回 502，目标连接超时时返回 504。该设计将请求解析、上游连接和响应回传组成完整的 HTTP 代理链路。

本功能对应的请求解析、报文改写和上游转发核心代码见【附录 A.1】。

3.2 访问控制策略链原理

访问控制采用有序、短路式策略链。每项策略返回“允许”或“拒绝”以及稳定的原因字段。只要某项策略拒绝请求，后续策略和上游连接立即停止，代理生成拦截响应并记录原因。该方式能够减少无效网络访问，也便于统计不同类型的拦截。

客户端访问控制根据 TCP 连接的源地址判断谁可以使用代理。IP 和 CIDR 网段通过网络前缀计算判断成员关系，黑名单优先于白名单；白名单为空时不限制客户端，白名单非空时只允许匹配项。代理认证检查 Proxy-Authorization 中的 Basic 凭据；固定时间窗口限流为每个客户端 IP 保存计数和重置时刻。

目标访问控制依次检查请求方法、白名单模式、域名黑名单和 URL 关键词。域名匹配支持完

整域名、父域名后缀和显式通配符。URL 关键词检查目标主机、路径和查询参数。策略原因使用 `domain_blacklist`、`client_ip_blacklist`、`url_keyword:...` 等稳定字段，使前端、日志和测试能够使用相同语义。

本功能对应的域名、客户端 IP/CIDR 和目标策略判定核心代码见【附录 A.2】。

3.3 网页正文关键词过滤原理

URL 过滤检查客户端发出的请求地址，正文过滤检查目标服务器返回的内容，两者位于处理流程的不同阶段。正文过滤首先分离 HTTP 响应头和响应体，再读取 `Content-Type` 与 `Content-Encoding`。系统只检查 HTML、纯文本、CSS、JavaScript 和 JSON 等文本类型；二进制文件和压缩响应直接放行，避免把任意字节错误解释为文本。

对于可检查的响应，系统根据响应头声明的字符集解码正文，并兼容 UTF-8 与 GB18030；若响应使用 `chunked` 传输，系统先还原各数据块后再匹配，使跨块关键词也能被识别。正文统一转换为小写后与规则逐项比较。命中关键词时，原始网页正文不会发送给客户端，代理改为生成 `403 Forbidden` 提示页。提示页包含拦截原因、命中规则、请求方法、目标地址和客户端地址，因此浏览器中显示的内容能够直接说明过滤结果。

正文过滤仅适用于代理能够读取的明文 HTTP 响应。若网站使用 HTTPS，浏览器和网站之间的应用层数据经过 TLS 加密，普通 CONNECT 代理无法读取网页正文。

本功能对应的分块解析、字符集解码和正文规则匹配核心代码见【附录 A.3】。

3.4 HTTPS CONNECT 隧道原理

浏览器访问 HTTPS 网站时，先向代理发送 `CONNECT` 请求，请求中包含目标域名和 443 端口。代理可以在建立隧道前根据目标域名执行访问控制。目标允许访问时，代理连接目标服务器，向浏览器返回 `200 Connection Established`，随后只负责双向转发加密字节。

系统将客户端 `Socket` 和上游 `Socket` 设置为非阻塞状态，使用 `select` 监视两端是否存在可读数据。客户端数据到达时转发给网站，网站数据到达时转发给客户端，并在连接关闭或长时间无活动时结束隧道。

`CONNECT` 隧道没有解密 TLS。代理能够看到目标域名和端口，看不到 HTTPS 内部路径、查询参数和正文。因此 HTTPS 域名拦截能够生效，HTTPS URL 路径和正文关键词过滤需要额外实现中间人解密和证书信任管理，不属于本项目的实现范围。

本功能对应的 `CONNECT` 双向转发核心代码见【附录 A.4】。

3.5 缓存、认证与限流设计

HTTP GET 缓存以“目标主机、端口和路径”组成缓存键，每个条目保存响应字节、状态码、创建时间和过期时间。请求到达时先检查条目是否存在且未过期；命中时仍按照当前正文规则重新检查缓存响应，未命中时连接上游。正文规则或缓存参数变化时系统清空旧缓存，避免规则热更新后继续返回旧网页。缓存容量达到上限时删除最早创建的条目。

代理认证使用 HTTP Basic 机制。认证失败时返回 `407` 和 `Proxy-Authenticate` 响应头；认证成功后才进入限流和目标策略。转发请求时删除代理认证头，保证目标网站不会收到代理账号密码。

限流采用固定时间窗口算法。每个客户端 IP 对应一个计数桶，桶内保存当前请求数和窗口重置时间。窗口内计数达到上限后返回 `429`，并记录剩余等待时间。认证挑战不消耗已认证请求额度，修改限流参数时清空旧计数桶，便于运行期间重新配置。

缓存、代理认证和固定时间窗口限流的核心代码见【附录 A.5】。

3.6 动态规则管理与并发一致性

规则可以通过管理前端、JSON API 或命令行工具修改。三种入口最终调用同一组后端规则操作，包括新增、删除、修改、查询和整组替换。后端先校验规则组，再按规则类型标准化输入。例如请求方法统一为大写，IP/CIDR 由 `ipaddress` 校验并转换为标准网络地址。

规则修改在运行状态对象的线程锁内完成。后端先更新内存配置，再把完整配置写回当前 JSON 文件，同时追加一条 JSONL 变更记录。新的客户端请求会读取最新配置，因此规则不需要重启代理即可生效。锁保证多个代理线程和管理线程并发工作时不会同时破坏共享配置、统计或缓存数据。

规则增删改查、缓存一致性处理和变更记录核心代码见【附录 A.6】。

3.7 日志审计与管理前端设计

日志按照用途分为访问日志、拦截日志、错误日志和规则变更记录。每条访问事件包含时间、事件类型、客户端地址、请求方法、目标主机、路径、状态码和字节数。拦截事件额外记录策略原因，规则变更记录包含操作类型、规则组、旧值、新值和保存结果。

日志查询模块把文本记录解析为结构化字段，支持按日志类别、事件类型和搜索关键词查询，并能够导出 CSV。运行状态对象维护总请求、放行、各类拦截、正文过滤、HTTPS 隧道、缓存、认证、限流和字节数统计。管理前端通过 JSON API 获取这些数据，提供规则编辑、统计展示、日志详情和规则变更回显。批量测试使用独立证据目录，前端可以读取最近一次验收结果，避免测试日志覆盖当前运行数据。

日志写入、结构化查询与 CSV 导出的核心代码见【附录 A.7】。

4 测试方案与验收结果

4.1 批量验证式测试

项目首先使用批量自动验证检查完整功能，再进行浏览器和命令行手工验收。自动测试为每个模块创建独立端口、测试配置和上游目标，并断言 HTTP 状态、响应内容、配置保存、统计与日志。Web/代理批次与规则/运行模式批次分别使用 `tests/evidence/web` 和 `tests/evidence/rules` 保存证据，重复执行不会依赖正式配置中的已有规则，也不会相互覆盖日志。

测试批次	覆盖内容	结果
Web/代理功能	HTTP 转发、域名拦截、URL 与方法过滤、正文过滤、CONNECT、缓存、前端、认证、限流、浏览器代理和验收证据	15/15
规则与运行模式	管理 API、白名单模式、规则增删改查、CLI、运行设置、客户端 IP/CIDR 与规则变更证据	7/7

批量验证命令如下：

```
python tests\run_all_smoke.py
```

测试通过时输出 `all test batches passed`。管理前端选择“最近验收”后，能够从持久化证据重建总请求、总拦截、各类统计、日志和规则变更记录。

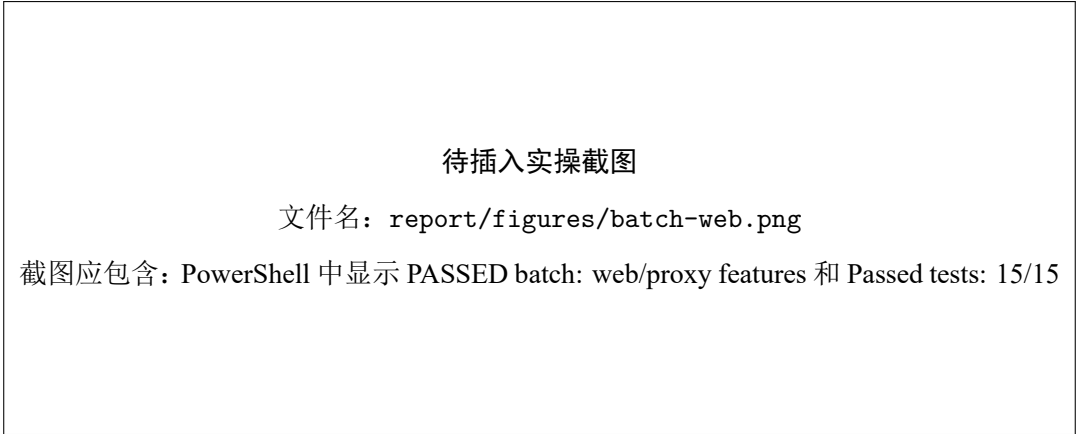


图 3 Web/代理功能批量验证结果

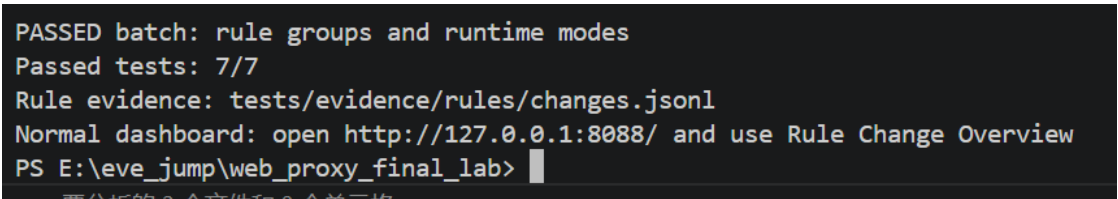


图 4 规则与运行模式批量验证结果



图 5 Web 批量验收结果在管理首页中的展示



图 6 规则批量验收变更总览

4.2 手工验收准备

手工验收用于观察浏览器页面、命令行状态码、管理前端统计和日志之间是否一致。URL 关键词和正文关键词使用真实明文 HTTP 网站，因为普通 CONNECT 代理无法读取 HTTPS 路径和正文。验收前启动代理：

```
python src\proxy.py --config config.example.json
```

验收浏览器可使用系统代理，也可以使用项目提供的独立配置。独立浏览器关闭自动 HTTPS 升级并降低浏览器缓存干扰：

```
powershell -ExecutionPolicy Bypass -File tools\start_proxy_browser.ps1
```

进行普通转发、域名、URL 和正文验收前，先关闭认证和限流干扰并切换为黑名单模式：

```
python tools\rule_cli.py set mode blacklist
python tools\rule_cli.py set proxy_auth_enabled false
python tools\rule_cli.py set rate_limit_enabled false
python tools\rule_cli.py rate-reset
```

4.3 普通转发与 HTTPS 隧道验收

首先在未添加 Bing 域名规则时访问 <https://www.bing.com/>。浏览器向代理发送 CONNECT 请求，代理建立到目标网站的 TCP 连接并双向转发 TLS 字节流。页面正常显示，同时日志产生 CONNECT 记录，说明代理具备真实网站访问转发能力。

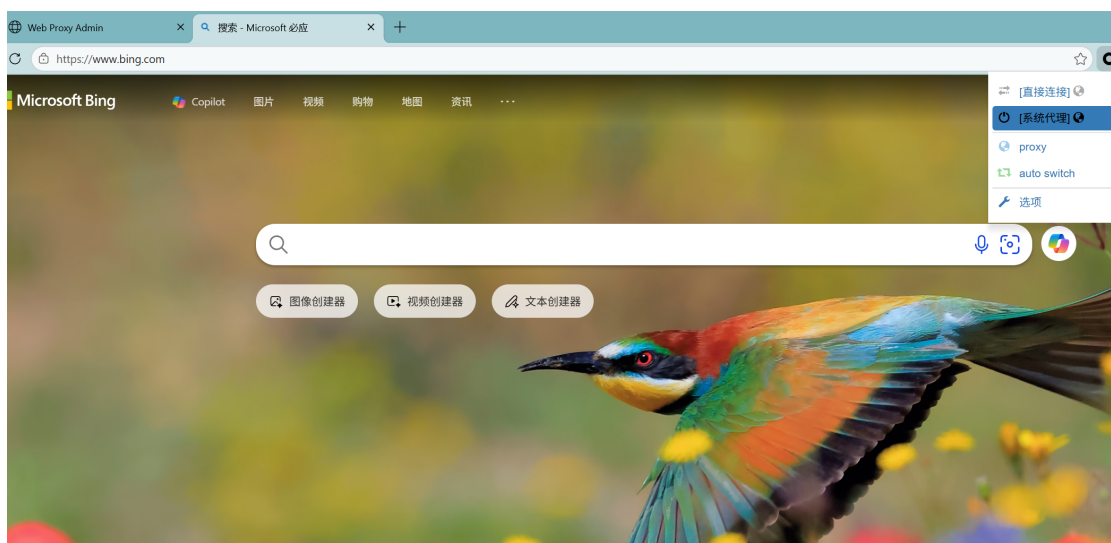


图 7 真实 HTTPS 网站通过代理正常访问

4.4 真实网站域名拦截验收

添加 Bing 子域名黑名单并访问真实 HTTPS 网站：

```
python tools\rule_cli.py add blocked_domains *.bing.com
```

在 Edge 中访问 <https://www.bing.com/>。浏览器会向代理发送 CONNECT `www.bing.com:443`，域名命中后代理拒绝建立 TLS 隧道。CONNECT 的 403 响应属于隧道协商失败，Edge 通常显示 `ERR_TUNNEL_CONNECTION_FAILED`，不会把代理返回的 HTML 403 正文渲染成网页。管理日志中的 `BLOCK reason=domain_blacklist` 用于确认具体原因。验收结束后删除规则：

```
python tools\rule_cli.py delete blocked_domains *.bing.com
```



图 8 真实 HTTPS 网站域名拦截结果



图 9 真实 HTTPS 域名拦截日志

4.5 真实网站 URL 关键词验收

URL 关键词验收使用真实明文 HTTP 地址 `http://httpforever.com/url-filter-demo`。该路径包含稳定关键词 `url-filter-demo`，代理在连接目标网站之前即可完成判断：

```
python tools\rule_cli.py delete blocked_domains httpforever.com
python tools\rule_cli.py add blocked_url_keywords url-filter-demo
```

在专用 Edge 中输入完整地址 `http://httpforever.com/url-filter-demo`。预期显示代理生成的 403 页面，详细信息中的拦截原因为 `url_keyword:url-filter-demo`。验收结束后删除规则：

```
python tools\rule_cli.py delete blocked_url_keywords url-filter-demo
```



图 10 真实网站 URL 关键词拦截结果

4.6 真实网站正文关键词验收

正文关键词验收使用真实明文 HTTP 网站 <http://neverssl.com/>。该网站首页正文包含 NeverSSL，代理连接网站并收到响应后才能判断正文是否命中。验收前清空代理缓存，查询参数用于避免 Edge 使用浏览器自身缓存：

```
python tools\rule_cli.py cache-clear
python tools\rule_cli.py delete blocked_domains neverssl.com
python tools\rule_cli.py delete blocked_url_keywords neverssl
python tools\rule_cli.py add blocked_content_keywords NeverSSL
```

在专用 Edge 中输入完整地址 <http://neverssl.com/?proxy-lab=content-check>。预期原网页正文被代理生成的 403 页面替代，详细信息中的拦截原因为 content_keyword:NeverSSL。验收结束后删除规则：

```
python tools\rule_cli.py delete blocked_content_keywords NeverSSL
```

待插入实操截图

文件名: report/figures/manual-content-neverssl.png

截图应包含: Edge 地址栏中的 <http://neverssl.com/?proxy-lab=content-check>，正文过滤后的 403 页面，以及展开后的 content_keyword:NeverSSL

图 11 真实网站正文关键词过滤结果

4.7 缓存、认证、限流与客户端地址策略验收

缓存验收先启用缓存并清空旧条目，随后连续访问同一个 HTTP GET 地址。第一次日志应为 CACHE_MISS 与 ALLOW，第二次应为 CACHE_HIT。正文规则发生变化时，系统自动清空旧缓存，防止旧网页绕过新规则。

```
python tools\rule_cli.py set cache_enabled true
python tools\rule_cli.py cache-clear
```

待插入实操截图

文件名: report/figures/manual-cache-log.png

截图应包含: 同一目标先出现 CACHE_MISS 和 ALLOW, 随后出现 CACHE_HIT

图 12 HTTP GET 缓存验收日志

代理认证验收启用 Basic 认证。无凭据请求应返回 407, 正确凭据请求能够继续访问, 且 Proxy-Authorization 不转发给目标网站。

待插入实操截图

文件名: report/figures/manual-auth.png

截图应包含: 未认证请求的 407 响应和带正确代理账号请求的正常响应

图 13 代理认证验收结果

限流验收将窗口内请求上限设为 1, 并在测试前重置计数。第一次请求记录 RATE_ALLOW, 第二次请求返回 429 并记录 RATE_LIMIT。浏览器会产生附加资源请求, 因此浏览器页面可能在主请求放行后很快出现限流失败; 命令行连续请求能更准确展示第一次放行、第二次拒绝。



图 14 浏览器访问频率限制结果

限流拦截日志

按日志类型、事件与关键字查询代理审计记录

返回管理平台

日志类型	事件类型，逗号分隔	全文查询	最大结果数	
拦截日志	RATE_LIMIT	例如 domain_blacklist 或目标域名	200	查询 导出 CSV
数据来源: 当前运行 日志类型: blocked 原始记录: 1097 匹配记录: 521 事件条件: RATE_LIMIT 全文条件: 无				
时间	事件	客户端	目标	详细信息
2026-06-12 18:13:51	RATE_LIMIT	127.0.0.1	cn.bing.com	client=127.0.0.1 method=CONNECT host=cn.bing.com count=20 limit=20 retry_after=3
2026-06-12 18:13:50	RATE_LIMIT	127.0.0.1	cn.bing.com	client=127.0.0.1 method=CONNECT host=cn.bing.com count=20 limit=20 retry_after=3
2026-06-12 18:13:50	RATE_LIMIT	127.0.0.1	client-side-detection.api.cx.metamask.io	client=127.0.0.1 method=CONNECT host=client-side-detection.api.cx.metamask.io count=20 limit=20 retry_after=3
2026-06-12 18:13:49	RATE_LIMIT	127.0.0.1	cn.bing.com	client=127.0.0.1 method=CONNECT host=cn.bing.com count=20 limit=20 retry_after=4

图 15 访问频率限制日志

客户端 IP 黑白名单验收使用 127.0.0.1 或对应 CIDR。黑名单命中时返回 403，白名单非空且客户端未命中时同样拒绝访问。

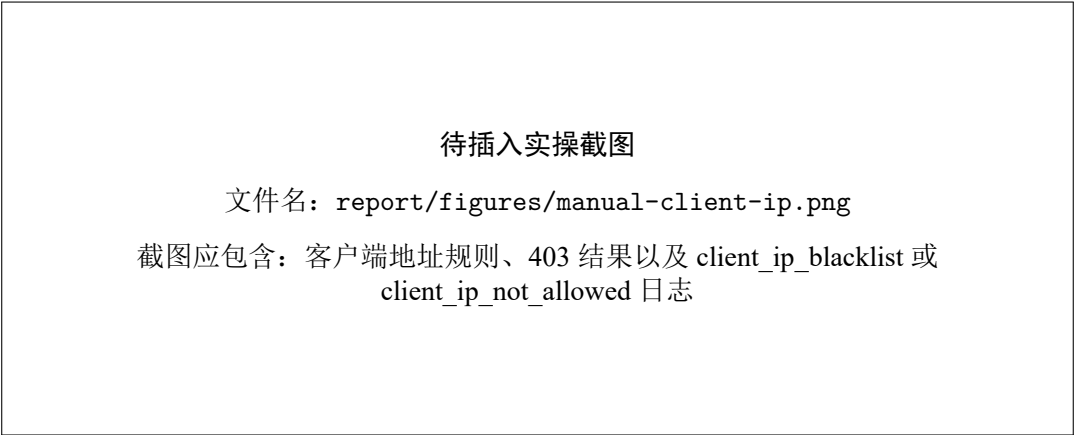


图 16 客户端 IP 黑白名单验收结果

4.8 日志、统计与规则变更验收

完成真实网站和运行设置验收后，管理首页“当前运行”数据源应显示本次总请求、总拦截和各分类统计。日志详情页支持事件与全文查询，规则变更总览默认读取当前运行历史；切换到“批量验收”后读取隔离的规则测试证据。



图 17 真实网站拦截日志查询结果

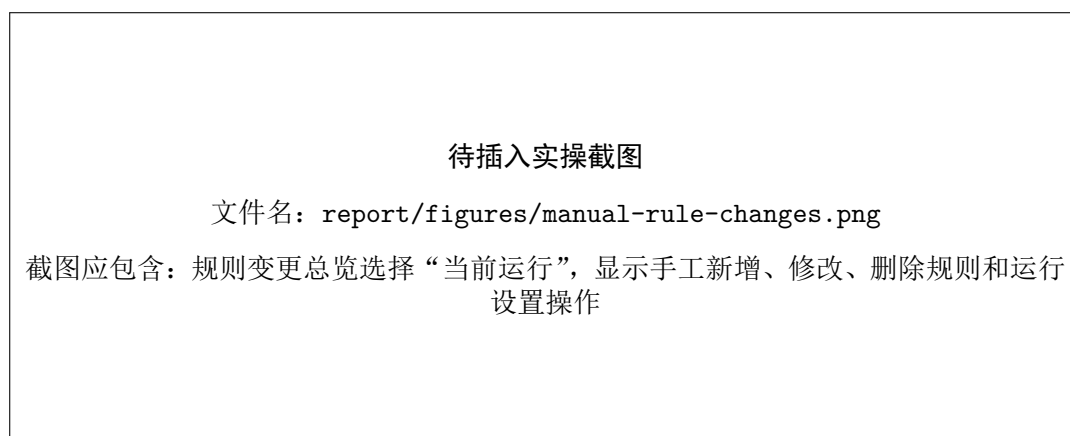


图 18 当前运行规则变更记录

4.9 测试结果分析

两组批量测试最终分别通过 15/15 和 7/7，说明代理转发、访问控制、正文过滤、CONNECT、缓存、认证、限流、客户端地址策略、动态规则管理、前端统计和审计证据能够在独立环境中重复执行。浏览器端回归测试曾因统计字段名称写错而在完成 13 项后失败，修正为实际字段 `filtered_keyword` 后完整批次通过；该过程说明测试脚本本身也需要接受回归检查。

真实网站手工验收体现了 HTTP 与 HTTPS 的可见性差异。明文 HTTP URL 规则能够在建立上游连接前返回带原因的 403 页面，正文规则能够在接收响应后替换原网页；HTTPS 域名拦截发生在 CONNECT 阶段，Edge 将其显示为隧道连接失败，具体 403 原因需要结合代理日志确认。

正文过滤故障排查发现，旧缓存曾在规则新增后直接返回原网页。修复后，正文规则与缓存参数变化会清空旧缓存，缓存命中也会按照当前正文规则重新检查；同时增加 `chunked` 正文还原和 GB18030 字符集测试。限流日志表明浏览器后台连接会共同消耗同一客户端 IP 的额度，因此现场演示其他模块前应关闭限流，单独验收限流时再启用并重置计数。

5 项目开发总结

5.1 完成情况

本项目完成了题目规定的 Web 请求转发、指定域名拦截和网页关键词过滤功能。代理能够解析 HTTP 代理请求并连接目标服务器，能够在连接目标服务器之前执行域名、URL、方法和客户端地址策略，也能够在收到明文 HTTP 响应后执行正文关键词过滤。

在基础功能之上，项目实现了 HTTPS CONNECT、GET 缓存、代理认证、访问频率限制、黑白名单模式、动态规则管理、管理前端、统计与日志审计。规则可以通过前端、API 和命令行修改，并立即作用于后续请求。自动测试与真实外网手工验收能够从响应、日志、统计和规则变更多个角度说明功能结果。

5.2 存在问题与改进方向

当前代理不解密 HTTPS 流量，因此无法检查 HTTPS 路径和正文。正文过滤已经支持 UTF-8、GB18030 和 chunked 文本响应；压缩响应与大文件流式过滤仍可继续完善。缓存和统计保存在进程内存中，重启后会清空；日志使用文件保存，在数据量较大时查询效率有限。

后续可以增加管理端登录、规则快照与回滚、SQLite 审计存储、定时规则和更完整的 HTTP 流式处理。HTTPS 内容检查需要在受控实验环境中设计证书生成、客户端信任和私钥保护机制，并明确其安全与合规边界。

6 心得体会

本项目的主要收获是将 HTTP 协议、TCP Socket 和访问控制策略结合成一个可以实际运行的系统。实现代理转发后，可以直接观察浏览器请求经过代理、被改写、转发并返回的过程；实现域名与正文过滤后，可以区分“请求发送前拒绝”和“收到响应后过滤”两种处理位置。

开发过程中，浏览器对本机地址的代理绕过、HTTPS CONNECT 的加密边界、PowerShell 与 CMD 的 JSON 引号差异，以及限流对浏览器附加请求的影响，都说明网络程序的验证需要结合协议行为和运行环境。通过为每个模块设计独立测试、日志原因和恢复命令，项目最终形成了可重复验证的完整实验流程。

参考文献

- [1] FIELDING R, NOTTINGHAM M, RESCHKE J. HTTP Semantics: RFC 9110[S]. Internet Engineering Task Force, 2022.
- [2] THOMSON M, NOTTINGHAM M. HTTP/1.1: RFC 9112[S]. Internet Engineering Task Force, 2022.
- [3] RESCORLA E. The Transport Layer Security (TLS) Protocol Version 1.3: RFC 8446[S]. Internet Engineering Task Force, 2018.
- [4] RESCHKE J. The “Basic” HTTP Authentication Scheme: RFC 7617[S]. Internet Engineering Task Force, 2015.
- [5] Python Software Foundation. socket—Low-level networking interface[EB/OL]. <https://docs.python.org/3/library/socket.html>.
- [6] Python Software Foundation. ipaddress—IPv4/IPv6 manipulation library[EB/OL]. <https://docs.python.org/3/library/ipaddress.html>.

A 关键源代码（带注释）

正文重点说明功能原理与设计。本附录列出与核心原理对应的实际项目代码，代码中的注释用于说明关键判断和安全边界。

A.1 HTTP 请求解析、改写与转发

Listing 1 HTTP 代理请求解析

```

1 def parse_http_request(request_text):
2     """解析客户端发来的 HTTP/代理请求，提取方法、目标主机、端口、路径和请求头。"""
3     lines = request_text.splitlines()
4     if not lines:
5         raise ValueError("empty request")
6
7     parts = lines[0].split()
8     if len(parts) != 3:
9         raise ValueError(f"invalid request line: {lines[0]}")
10
11     method, target, version = parts
12     host_header = find_header(lines[1:], "Host")
13
14     if method.upper() == "CONNECT":
15         host_part = target
16         if not host_part and host_header:
17             host_part = host_header
18         if ":" in host_part:
19             host, port_text = host_part.rsplit(":", 1)
20             port = int(port_text)
21         else:
22             host = host_part
23             port = 443
24         path = target
25     elif target.startswith("http://"):
26         parsed = urlsplit(target)
27         host = parsed.hostname
28         port = parsed.port or 80
29         path = parsed.path or "/"
30         if parsed.query:
31             path += "?" + parsed.query
32     else:
33         if not host_header:
34             raise ValueError("missing Host header")
35         host_part = host_header
36         if ":" in host_part:
37             host, port_text = host_part.rsplit(":", 1)
38             port = int(port_text)
39         else:
40             host = host_part
41             port = 80
42         path = target or "/"
43
44     if not host:
45         raise ValueError("missing target host")
46
47     return {
48         "method": method.upper(),
49         "target": target,
50         "version": version,
51         "host": host,
52         "port": port,
53         "path": path,
54         "host_header": host_header,
55         "headers": {
56             line.split(":", 1)[0].strip().lower(): line.split(":", 1)[1].strip()
57             for line in lines[1:]
58             if ":" in line
59         },
60     }

```

Listing 2 上游请求改写、连接与响应转发

```

1 def build_upstream_request(request_text, request_info):
2     """把代理格式请求改写成真实服务器能接受的普通 HTTP 请求。"""
3     header_lines, body = split_request(request_text)
4     upstream_lines = [
5         f"{request_info['method']} {request_info['path']} {request_info['version']}",
6         f"Host: {request_info['host']}:{request_info['port']}"
7         if request_info["port"] != 80
8         else f"Host: {request_info['host']}",
9     ]
10
11     skipped_headers = {
12         "host",
13         "connection",
14         "proxy-connection",
15         # 代理认证凭据只能由本代理读取，不能转发给目标网站。
16         "proxy-authorization",
17         "proxy-authenticate",
18         "accept-encoding",
19     }
20
21     for line in header_lines[1:]:
22         if ":" not in line:
23             continue
24         header_name = line.split(":", 1)[0].strip().lower()
25         if header_name in skipped_headers:
26             continue
27         upstream_lines.append(line)
28
29     upstream_lines.append("Connection: close")
30     upstream_lines.append("Accept-Encoding: identity")
31
32     upstream_text = "\r\n".join(upstream_lines) + "\r\n\r\n" + body
33     return upstream_text.encode("iso-8859-1", errors="replace")
34
35
36 def forward_http(client_socket, request_text, request_info, config, client_ip=""):
37     """处理普通 HTTP 请求：连接目标服务器、转发请求、接收响应、执行正文过滤。"""
38     upstream_request = build_upstream_request(request_text, request_info)
39     timeout = int(config.get("timeout_seconds", DEFAULT_TIMEOUT))
40
41     try:
42         with socket.create_connection(
43             (request_info["host"], request_info["port"]),
44             timeout=timeout,
45         ) as upstream_socket:
46             upstream_socket.settimeout(timeout)
47             upstream_socket.sendall(upstream_request)
48
49             chunks = []
50             while True:
51                 chunk = upstream_socket.recv(BUFFER_SIZE)
52                 if not chunk:
53                     break
54                 chunks.append(chunk)
55
56             response_bytes = b"".join(chunks)
57             status_code = parse_status_code(response_bytes)
58             response_bytes, blocked_keyword = filter_response_content(
59                 response_bytes,
60                 config,

```

```

61         request_info,
62         client_ip,
63     )
64     if blocked_keyword:
65         status_code = 403
66     client_socket.sendall(response_bytes)
67     return {
68         "outcome": "filtered" if blocked_keyword else "allowed",
69         "status_code": status_code,
70         "bytes_sent": len(response_bytes),
71         "keyword": blocked_keyword,
72         "response_bytes": response_bytes,
73     }
74 except socket.timeout:
75     bytes_sent = send_simple_response(
76         client_socket,
77         504,
78         "Gateway Timeout",
79         "target server timed out",
80     )
81     return {"outcome": "error", "status_code": 504, "bytes_sent": bytes_sent}
82 except OSError as error:
83     bytes_sent = send_simple_response(
84         client_socket,
85         502,
86         "Bad Gateway",
87         f"cannot connect to target server: {error}",
88     )
89     return {"outcome": "error", "status_code": 502, "bytes_sent": bytes_sent}

```

A.2 访问控制策略判定

Listing 3 域名、客户端 IP/CIDR 与目标访问策略

```

1 def domain_matches(host, patterns):
2     """匹配域名规则，支持完整域名、后缀域名和显式通配符。"""
3     host = host.lower().strip(".")
4     for pattern in patterns:
5         pattern = pattern.lower().strip(".")
6         if not pattern:
7             continue
8         # 显式通配符由 * 表达，例如 *.baidu.com 或 *baidu*。
9         # 不再把普通 baidu 当成“任意包含 baidu”，避免误拦其他域名。
10        if fnmatch.fnmatch(host, pattern):
11            return True
12        if "*" not in pattern and (host == pattern or host.endswith("." + pattern)):
13            return True
14    return False
15
16
17 def client_ip_matches(client_ip, patterns):
18     """判断客户端地址是否命中单个 IP 或 CIDR 网段规则。"""
19     try:
20         address = ipaddress.ip_address(client_ip)
21     except ValueError:
22         return False
23     for pattern in patterns:
24         try:
25             if address == ipaddress.ip_address(pattern):
26                 return True
27             continue
28         except ValueError:

```

```

29         pass
30     try:
31         if address in ipaddress.ip_network(pattern, strict=False):
32             return True
33     except ValueError:
34         # 手工修改配置时出现的非法条目会被安全忽略。
35         continue
36     return False
37
38
39 def check_client_access_policy(client_ip, config):
40     """在认证和目标过滤前执行客户端 IP 黑白名单策略。"""
41     blocked_clients = config.get("blocked_client_ips", [])
42     if client_ip_matches(client_ip, blocked_clients):
43         return False, "client_ip_blacklist"
44
45     allowed_clients = config.get("allowed_client_ips", [])
46     if allowed_clients and not client_ip_matches(client_ip, allowed_clients):
47         return False, "client_ip_not_allowed"
48     return True, "allow"
49
50
51 def check_access_policy(request_info, config):
52     """执行访问控制：方法黑名单、白名单模式、域名黑名单和 URL 关键字拦截。"""
53     host = request_info["host"]
54     method = request_info["method"]
55     path = request_info["path"]
56     target = request_info["target"]
57     mode = config.get("mode", "blacklist").lower()
58
59     blocked_methods = [item.upper() for item in config.get("blocked_methods", [])]
60     if method in blocked_methods:
61         return False, "method_blacklist"
62
63     if mode == "whitelist":
64         allowed_domains = config.get("allowed_domains", [])
65         if not domain_matches(host, allowed_domains):
66             return False, "domain_not_in_whitelist"
67
68     blocked_domains = config.get("blocked_domains", [])
69     if domain_matches(host, blocked_domains):
70         return False, "domain_blacklist"
71
72     searchable_url = f"{host}{path} {target}".lower()
73     for keyword in config.get("blocked_url_keywords", []):
74         if keyword.lower() in searchable_url:
75             return False, f"url_keyword:{keyword}"
76
77     return True, "allow"

```

A.3 HTTP 明文正文关键词过滤

Listing 4 分块解析、字符集解码与正文关键词过滤

```

1 def decode_chunked_body(body):
2     """解析 HTTP chunked 正文，仅用于关键词检查；解析失败时保留原始正文。"""
3     chunks = []
4     position = 0
5     try:
6         while True:
7             line_end = body.find(b"\r\n", position)
8             if line_end < 0:

```

```

9         return body
10        size_text = body[position:line_end].split(b";", 1)[0].strip()
11        size = int(size_text, 16)
12        position = line_end + 2
13        if size == 0:
14            return b"".join(chunks)
15        chunk_end = position + size
16        if chunk_end + 2 > len(body) or body[chunk_end:chunk_end + 2] != b"\r\n":
17            return body
18        chunks.append(body[position:chunk_end])
19        position = chunk_end + 2
20    except ValueError:
21        return body
22
23
24    def decode_text_body(body, content_type):
25        """按响应声明字符集解码正文，并兼容常见 UTF-8、GB18030 页面。"""
26        declared_charset = ""
27        for parameter in content_type.split(";")[1:]:
28            name, separator, value = parameter.partition("=")
29            if separator and name.strip().lower() == "charset":
30                declared_charset = value.strip().strip("\'")
31            break
32
33        encodings = [declared_charset, "utf-8", "gb18030", "iso-8859-1"]
34        tried = set()
35        for encoding in encodings:
36            normalized = encoding.lower()
37            if not normalized or normalized in tried:
38                continue
39            tried.add(normalized)
40            try:
41                return body.decode(encoding)
42            except (LookupError, UnicodeDecodeError):
43                continue
44        return body.decode("utf-8", errors="replace")
45
46
47    def filter_response_content(response_bytes, config, request_info=None, client_ip=""):
48        """对 HTTP 明文文本响应做正文关键字过滤，二进制或压缩内容直接放行。"""
49        if b"\r\n\r\n" not in response_bytes:
50            return response_bytes, None
51
52        header_bytes, body = response_bytes.split(b"\r\n\r\n", 1)
53        header_text = header_bytes.decode("iso-8859-1", errors="replace")
54        header_lines = header_text.splitlines()
55
56        content_type = response_header_value(header_lines, "Content-Type")
57        content_encoding = response_header_value(header_lines, "Content-Encoding")
58        transfer_encoding = response_header_value(header_lines, "Transfer-Encoding")
59
60        if content_encoding and content_encoding.lower() not in ("identity", "none"):
61            return response_bytes, None
62
63        if not is_text_content_type(content_type):
64            return response_bytes, None
65
66        inspection_body = decode_chunked_body(body) if "chunked" in transfer_encoding.lower() else body
67        body_text = decode_text_body(inspection_body, content_type)
68        body_text_lower = body_text.lower()
69
70        for keyword in config.get("blocked_content_keywords", []):

```

```

71         if keyword.lower() in body_text_lower:
72             return (
73                 build_policy_block_response(
74                     f"content_keyword:{keyword}",
75                     request_info,
76                     client_ip,
77                     detail=keyword,
78                 ),
79                 keyword,
80             )
81
82     return response_bytes, None
83
84
85 def split_request(request_text):
86     if "\r\n\r\n" in request_text:
87         header_text, body = request_text.split("\r\n\r\n", 1)
88     elif "\n\n" in request_text:
89         header_text, body = request_text.split("\n\n", 1)
90     else:
91         header_text, body = request_text, ""
92     return header_text.splitlines(), body

```

A.4 HTTPS CONNECT 双向隧道

Listing 5 HTTPS CONNECT 隧道双向转发

```

1  def forward_connect(client_socket, request_info, config):
2      """处理 HTTPS CONNECT: 建立 TCP 隧道, 只转发加密字节流, 不读取 HTTPS 正文。"""
3      timeout = int(config.get("timeout_seconds", DEFAULT_TIMEOUT))
4      response = b"HTTP/1.1 200 Connection Established\r\nConnection: close\r\n\r\n"
5      bytes_to_client = 0
6      bytes_from_client = 0
7
8      try:
9          with socket.create_connection(
10              (request_info["host"], request_info["port"]),
11              timeout=timeout,
12          ) as upstream_socket:
13              client_socket.sendall(response)
14              bytes_to_client += len(response)
15
16              client_socket.setblocking(False)
17              upstream_socket.setblocking(False)
18
19              sockets = [client_socket, upstream_socket]
20              last_activity = time.time()
21
22              while True:
23                  if time.time() - last_activity > timeout:
24                      break
25
26                  readable, _, exceptional = select.select(sockets, [], sockets, 1)
27                  if exceptional:
28                      break
29
30                  for ready_socket in readable:
31                      try:
32                          data = ready_socket.recv(BUFFER_SIZE)
33                      except BlockingIOError:
34                          continue
35

```

```

36         if not data:
37             return {
38                 "outcome": "tunnel",
39                 "status_code": 200,
40                 "bytes_sent": bytes_to_client,
41                 "bytes_from_client": bytes_from_client,
42             }
43
44         last_activity = time.time()
45         if ready_socket is client_socket:
46             upstream_socket.sendall(data)
47             bytes_from_client += len(data)
48         else:
49             client_socket.sendall(data)
50             bytes_to_client += len(data)
51
52     return {
53         "outcome": "tunnel",
54         "status_code": 200,
55         "bytes_sent": bytes_to_client,
56         "bytes_from_client": bytes_from_client,
57     }
58 except socket.timeout:
59     bytes_sent = send_simple_response(
60         client_socket,
61         504,
62         "Gateway Timeout",
63         "target server timed out",
64     )
65     return {
66         "outcome": "error",
67         "status_code": 504,
68         "bytes_sent": bytes_sent,
69         "bytes_from_client": bytes_from_client,
70     }
71 except OSError as error:
72     bytes_sent = send_simple_response(
73         client_socket,
74         502,
75         "Bad Gateway",
76         f"cannot connect to target server: {error}",
77     )
78     return {
79         "outcome": "error",
80         "status_code": 502,
81         "bytes_sent": bytes_sent,
82         "bytes_from_client": bytes_from_client,
83     }

```

A.5 缓存、认证与固定时间窗口限流

Listing 6 GET 缓存与客户端限流桶

```

1 def get_cache(self, key):
2     """读取 HTTP GET 缓存; 如果缓存过期, 自动删除并返回 None."""
3     with self.lock:
4         entry = self.cache.get(key)
5         if not entry:
6             return None
7         if entry["expires_at"] < time.time():
8             del self.cache[key]
9             return None

```



```

10         return dict(entry)
11
12     def set_cache(self, key, response_bytes, status_code, ttl_seconds, max_items):
13         """写入 HTTP GET 缓存, 并在超过最大条目数时删除最早的缓存。"""
14         if ttl_seconds <= 0 or max_items <= 0:
15             return
16         with self.lock:
17             while max_items > 0 and len(self.cache) >= max_items:
18                 oldest_key = min(
19                     self.cache,
20                     key=lambda item: self.cache[item]["created_at"],
21                 )
22                 del self.cache[oldest_key]
23             self.cache[key] = {
24                 "response_bytes": response_bytes,
25                 "status_code": status_code,
26                 "created_at": time.time(),
27                 "expires_at": time.time() + ttl_seconds,
28             }
29
30     def check_rate_limit(self, client_ip, max_requests, window_seconds):
31         """按客户端 IP 做固定时间窗口限流, 返回是否允许和剩余/等待时间。"""
32         if max_requests <= 0:
33             return True, 0, 0
34         now = time.time()
35         with self.lock:
36             bucket = self.rate_limits.get(client_ip)
37             if not bucket or now >= bucket["reset_at"]:
38                 self.rate_limits[client_ip] = {
39                     "count": 1,
40                     "reset_at": now + window_seconds,
41                 }
42             return True, 0, 1
43             if bucket["count"] >= max_requests:
44                 retry_after = max(1, int(bucket["reset_at"] - now))
45                 return False, retry_after, bucket["count"]
46             bucket["count"] += 1
47             return True, 0, bucket["count"]

```

Listing 7 Proxy-Authorization Basic 代理认证

```

1 def check_proxy_auth(request_info, config):
2     """检查 Proxy-Authorization Basic 认证头, 认证关闭时直接放行。"""
3     if not config.get("proxy_auth_enabled", False):
4         return True
5
6     users = config.get("proxy_auth_users", {})
7     auth_header = request_info.get("headers", {}).get("proxy-authorization", "")
8     if not auth_header.lower().startswith("basic "):
9         return False
10
11     token = auth_header.split(None, 1)[1].strip()
12     try:
13         decoded = base64.b64decode(token).decode("utf-8")
14     except Exception:
15         return False
16
17     if ":" not in decoded:
18         return False
19     username, password = decoded.split(":", 1)
20     return users.get(username) == password
21

```

```

22
23 def is_cacheable_request(request_info, config):
24     return bool(config.get("cache_enabled", False)) and request_info["method"] == "GET"
25
26
27 def build_cache_key(request_info):
28     return f"{request_info['host']}:{request_info['port']}:{request_info['path']}"

```

A.6 动态规则与缓存一致性

Listing 8 规则变更记录、正文规则缓存清理与新增操作

```

1 def _record_change_locked(self, action, result):
2     """保存一条管理操作记录，调用方必须已经持有 self.lock。"""
3     entry = {
4         "time": datetime.now().isoformat(timespec="seconds"),
5         "action": action,
6         "changed": bool(result.get("changed", False)),
7         "saved": bool(result.get("saved", False)),
8     }
9     for key in (
10         "rule_type",
11         "value",
12         "old_value",
13         "new_value",
14         "values",
15         "settings",
16         "cache_cleared",
17     ):
18         if key in result:
19             entry[key] = result[key]
20     self.change_history.insert(0, entry)
21     self.change_history = self.change_history[:100]
22     self._append_change_history_locked(entry)
23     return entry
24
25 def _clear_cache_for_rule_change_locked(self, rule_type, changed):
26     """正文过滤规则变化后清空旧响应缓存，调用方必须持有 self.lock。"""
27     if changed and rule_type == "blocked_content_keywords":
28         cleared = len(self.cache)
29         self.cache.clear()
30         return cleared
31     return 0
32
33 def add_list_rule(self, rule_type, value):
34     """向指定规则列表新增一条规则，并立即保存到配置文件。"""
35     validate_rule_type(rule_type)
36     normalized = normalize_rule_value(rule_type, value)
37     new_identity = rule_identity(rule_type, normalized)
38     with self.lock:
39         current_values = list(self.config.get(rule_type, []))
40         current_identities = {
41             rule_identity(rule_type, item)
42             for item in current_values
43         }
44         if new_identity in current_identities:
45             changed = False
46         else:
47             current_values.append(normalized)
48             self.config[rule_type] = current_values
49             changed = True
50     saved = self._save_config_locked()

```

```

51         result = {
52             "ok": True,
53             "changed": changed,
54             "saved": saved,
55             "rule_type": rule_type,
56             "value": normalized,
57             "values": list(self.config.get(rule_type, [])),
58             "cache_cleared": self._clear_cache_for_rule_change_locked(rule_type, changed),
59         }
60         result["history_entry"] = self._record_change_locked("add", result)
61         return result

```

A.7 日志查询与 CSV 审计导出

Listing 9 日志写入、结构化查询与 CSV 导出

```

1  def log_event(state, event_type, message):
2      """打印并持久化事件，同时写入专用拦截或错误日志。"""
3      config = state.get_config()
4      timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
5      line = f"[{timestamp}] {event_type} {message}"
6      print(line, flush=True)
7
8      write_log_line(config, "log_file", line)
9      # 认证失败和限流拒绝同样属于拦截事件。
10     if event_type.startswith(("BLOCK", "FILTER", "AUTH_REQUIRED", "RATE_LIMIT")):
11         write_log_line(config, "blocked_log_file", line)
12     if event_type.startswith("ERROR"):
13         write_log_line(config, "error_log_file", line)
14
15
16 def read_log_tail(config, kind, limit):
17     """读取最新的访问、拦截或错误日志行。"""
18     file_key = LOG_FILE_KEYS.get(kind, "log_file")
19     log_path_text = config.get(file_key)
20     if not log_path_text:
21         return []
22     log_path = resolve_project_path(log_path_text)
23     if not log_path.exists():
24         return []
25     lines = log_path.read_text(encoding="utf-8", errors="replace").splitlines()
26     return lines[-limit:]
27
28
29 def parse_log_line(line):
30     """把文本日志解析成适合查询结果表格展示的结构化字段。"""
31     match = LOG_LINE_PATTERN.match(line)
32     if not match:
33         return {"time": "", "event": "UNKNOWN", "message": line, "fields": {}, "raw": line}
34
35     message = match.group("message") or ""
36     fields = {}
37     for token in message.split():
38         if "=" not in token:
39             continue
40         key, value = token.split("=", 1)
41         fields[key] = value
42     return {
43         "time": match.group("time"),
44         "event": match.group("event"),
45         "message": message,
46         "fields": fields,

```

```

47     "raw": line,
48 }
49
50
51 def query_log_entries(config, kind="proxy", events=None, search="", limit=200):
52     """像只读小型数据库一样筛选日志，并返回结构化记录。"""
53     normalized_kind = kind if kind in LOG_FILE_KEYS else "proxy"
54     file_key = LOG_FILE_KEYS[normalized_kind]
55     log_path_text = config.get(file_key)
56     if not log_path_text:
57         return {
58             "kind": normalized_kind,
59             "events": [],
60             "search": search,
61             "total": 0,
62             "matched": 0,
63             "entries": [],
64         }
65
66     log_path = resolve_project_path(log_path_text)
67     lines = (
68         log_path.read_text(encoding="utf-8", errors="replace").splitlines()
69         if log_path.exists()
70         else []
71     )
72     event_set = {event.strip().upper() for event in (events or []) if event.strip()}
73     search_lower = str(search or "").strip().lower()
74
75     matched_entries = []
76     for line in lines:
77         entry = parse_log_line(line)
78         if event_set and entry["event"] not in event_set:
79             continue
80         if search_lower and search_lower not in line.lower():
81             continue
82         matched_entries.append(entry)
83
84     bounded_limit = max(1, min(int(limit), 1000))
85     return {
86         "kind": normalized_kind,
87         "events": sorted(event_set),
88         "search": search,
89         "total": len(lines),
90         "matched": len(matched_entries),
91         # 详情页按时间倒序展示，便于先查看最新事件。
92         "entries": list(reversed(matched_entries[-bounded_limit:])),
93     }
94
95
96 def build_log_csv(query_result):
97     """把结构化查询结果序列化为适合 Excel 打开的 UTF-8 CSV。"""
98     text_buffer = io.StringIO(newline="")
99     writer = csv.DictWriter(text_buffer, fieldnames=CSV_FIELD_NAMES)
100     writer.writeheader()
101     for entry in query_result.get("entries", []):
102         fields = entry.get("fields", {})
103         # 常用审计字段单独成列，同时保留完整原始证据。
104         writer.writerow(
105             {
106                 "time": entry.get("time", ""),
107                 "event": entry.get("event", ""),
108                 "client": fields.get("client", ""),

```

```

109         "method": fields.get("method", ""),
110         "host": fields.get("host", ""),
111         "path": fields.get("path", ""),
112         "status": fields.get("status", ""),
113         "reason": fields.get("reason", ""),
114         "keyword": fields.get("keyword", ""),
115         "message": entry.get("message", ""),
116         "raw": entry.get("raw", ""),
117     }
118 )
119 # utf-8-sig 添加 BOM, 使 Windows Excel 能直接识别中文。
120 return text_buffer.getvalue().encode("utf-8-sig")

```

A.8 客户端连接的有序处理总入口

Listing 10 认证、限流、策略、缓存、转发与日志处理顺序

```

1 def handle_client(client_socket, client_address, state):
2     """处理单个客户端连接, 是认证、限流、规则过滤、缓存、转发的总入口。"""
3     config = state.get_config()
4     try:
5         data = client_socket.recv(BUFFER_SIZE)
6         if not data:
7             return
8
9         state.increment("total_requests")
10        state.increment("bytes_from_clients", len(data))
11
12        request_text = data.decode("iso-8859-1", errors="replace")
13        request_info = parse_http_request(request_text)
14        state.record_host(request_info["host"])
15
16        print("=" * 60, flush=True)
17        print(f"Client: {client_address[0]}:{client_address[1]}", flush=True)
18        print(
19            f"Request line: {request_info['method']} {request_info['target']} {request_info['version']}",
20            flush=True,
21        )
22        print(f"Host: {request_info['host_header']}", flush=True)
23        print(
24            "Parsed: "
25            f"method={request_info['method']} "
26            f"host={request_info['host']} "
27            f"port={request_info['port']} "
28            f"path={request_info['path']}",
29            flush=True,
30        )
31        print("=" * 60, flush=True)
32
33        # 客户端地址策略先于认证和限流执行, 拒绝的主机不会消耗认证与限流资源。
34        client_allowed, client_reason = check_client_access_policy(client_address[0], config)
35        if not client_allowed:
36            update_block_stats(state, client_reason)
37            bytes_sent = send_policy_block_response(
38                client_socket,
39                client_reason,
40                request_info,
41                client_address[0],
42            )
43            state.increment("bytes_to_clients", bytes_sent)
44            log_event(
45                state,

```

```

46         "BLOCK",
47         f"client={client_address[0]} method={request_info['method']} "
48         f"host={request_info['host']} path={request_info['path']} reason={client_reason}",
49     )
50     return
51
52 if not check_proxy_auth(request_info, config):
53     state.increment("auth_required")
54     bytes_sent = send_auth_required_response(client_socket)
55     state.increment("bytes_to_clients", bytes_sent)
56     log_event(
57         state,
58         "AUTH_REQUIRED",
59         f"client={client_address[0]} method={request_info['method']} host={request_info['host']}",
60     )
61     return
62
63 # 仅对通过认证的请求计数，避免 407 认证挑战消耗限流额度。
64 rate_config_enabled = config.get("rate_limit_enabled", False)
65 if rate_config_enabled:
66     max_requests = int(config.get("rate_limit_per_minute", 60))
67     window_seconds = int(config.get("rate_limit_window_seconds", 60))
68     allowed_by_rate, retry_after, current_count = state.check_rate_limit(
69         client_address[0],
70         max_requests,
71         window_seconds,
72     )
73     if not allowed_by_rate:
74         state.increment("rate_limited")
75         bytes_sent = send_simple_response(
76             client_socket,
77             429,
78             "Too Many Requests",
79             f"rate limit exceeded, retry after {retry_after} seconds",
80         )
81         state.increment("bytes_to_clients", bytes_sent)
82         log_event(
83             state,
84             "RATE_LIMIT",
85             f"client={client_address[0]} method={request_info['method']} "
86             f"host={request_info['host']} count={current_count} "
87             f"limit={max_requests} retry_after={retry_after}",
88         )
89         return
90     log_event(
91         state,
92         "RATE_ALLOW",
93         f"client={client_address[0]} method={request_info['method']} "
94         f"host={request_info['host']} count={current_count} limit={max_requests}",
95     )
96
97 allowed, reason = check_access_policy(request_info, config)
98 if not allowed:
99     update_block_stats(state, reason)
100     bytes_sent = send_policy_block_response(
101         client_socket,
102         reason,
103         request_info,
104         client_address[0],
105     )
106     state.increment("bytes_to_clients", bytes_sent)
107     log_event(

```

```

108         state,
109         "BLOCK",
110         f"client={client_address[0]} method={request_info['method']} "
111         f"host={request_info['host']} path={request_info['path']} reason={reason}",
112     )
113     return
114
115     cache_key = None
116     if is_cacheable_request(request_info, config):
117         cache_key = build_cache_key(request_info)
118         cached = state.get_cache(cache_key)
119         if cached:
120             # 缓存中保存的是上游原始响应。每次命中仍需按当前正文规则检查，
121             # 否则先访问页面、再新增正文规则时，浏览器会一直拿到旧缓存页面。
122             response_bytes, blocked_keyword = filter_response_content(
123                 cached["response_bytes"],
124                 config,
125                 request_info,
126                 client_address[0],
127             )
128             client_socket.sendall(response_bytes)
129             bytes_sent = len(response_bytes)
130             state.increment("cache_hits")
131             state.increment("bytes_to_clients", bytes_sent)
132             if blocked_keyword:
133                 state.increment("filtered_keyword")
134                 state.record_keyword(blocked_keyword)
135                 log_event(
136                     state,
137                     "CACHE_HIT_FILTERED",
138                     f"client={client_address[0]} method={request_info['method']} "
139                     f"host={request_info['host']} path={request_info['path']} "
140                     f"keyword={blocked_keyword}",
141                 )
142                 log_event(
143                     state,
144                     "FILTER",
145                     f"client={client_address[0]} method={request_info['method']} "
146                     f"host={request_info['host']} path={request_info['path']} "
147                     f"keyword={blocked_keyword}",
148                 )
149             return
150         state.increment("allowed_requests")
151         log_event(
152             state,
153             "CACHE_HIT",
154             f"client={client_address[0]} method={request_info['method']} "
155             f"host={request_info['host']} path={request_info['path']} "
156             f"status={cached.get('status_code', 0)} bytes={bytes_sent}",
157         )
158         return
159     state.increment("cache_misses")
160     log_event(
161         state,
162         "CACHE_MISS",
163         f"client={client_address[0]} method={request_info['method']} "
164         f"host={request_info['host']} path={request_info['path']}",
165     )
166
167     if request_info["method"] == "CONNECT":
168         result = forward_connect(client_socket, request_info, config)
169     else:

```

```

170         result = forward_http(
171             client_socket,
172             request_text,
173             request_info,
174             config,
175             client_address[0],
176         )
177
178     state.increment("bytes_to_clients", result.get("bytes_sent", 0))
179     state.increment("bytes_from_clients", result.get("bytes_from_client", 0))
180
181     if result["outcome"] == "allowed":
182         if cache_key and result.get("status_code") == 200 and result.get("response_bytes"):
183             state.set_cache(

```

B 常用运行与验收命令

命令	用途
<code>python src\proxy .py --config config.example.json</code>	启动代理服务和管理前端。
<code>python tests\run_all_smoke.py</code>	运行完整自动回归测试。
<code>python tools\rule_cli.py list</code>	查询所有规则组。
<code>python tools\rule_cli.py config</code>	查询当前运行配置。
<code>python tools\rule_cli.py changes --limit 20</code>	查询最近规则变更。
<code>python tools\rule_cli.py log-query --kind blocked --limit 20</code>	查询最近拦截日志。